AI Chatbot

A Data Science GenAI Fine-Tuning Project

Domain Knowledge Chatbot for Codebase
and Developer Expertise

Abhishek Srivastava
DSC 680 | Week 10
Bellevue University

GPT-3.5-Turbo | RAG | LangChain | Streamlit

The Business Problem

Software development teams lose significant time when developers cannot quickly find answers about their own codebase. When team members leave or switch projects, institutional knowledge — design decisions, coding conventions, architecture rationale — leaves with them.

60%

Developer Time

Spent searching for answers in scattered docs, Slack, and wikis

3-6

Weeks Onboarding

New developers spend weeks just understanding the existing codebase

80%

Repeated Questions

Senior devs answer the same questions again and again

Solution: A Domain Knowledge Chatbot that makes project knowledge searchable through natural language

Data Sources & RAG Architecture

5 Data Sources

- Git commit histories — what changed and why
- Source code files — ground-truth implementation
- Architecture & design documents — system structure
- API specifications — interface contracts & behaviors
- Knowledge transfer docs — curated onboarding info

RAG Pipeline Flow

1. Developer submits a natural language query
2. Query converted to vector embedding
3. Retrieve most relevant chunks from vector DB
4. Chunks passed as context in prompt to GPT
5. GPT generates a grounded, cited response

Preprocessing: Deduplication → Chunking → Embedding → Metadata Tagging

Fine-Tuning Strategy & Model

Model Configuration

Base Model: GPT-3.5-Turbo
Fine-tuning: OpenAI hosted API
Temperature: 0.3
Max Tokens: 500
Training Data: 300-500 Q&A pairs
Target Accuracy: 80% → 90%+

Training Data Sources

- Engineering Slack/Teams channels
- Code review threads
- Onboarding FAQs
- Incident post-mortems
- Architecture decision records

Two Approaches Implemented

Approach 1: RAG with OpenAI Embeddings

Ada-002 embeddings + semantic search + GPT completion

Approach 2: Fine-Tuned GPT-2

HuggingFace Trainer + domain-specific corpus fine-tuning

Prompt Experiments & Results

Experiment	Use Case	Result
Exp 1	Client setup & reusable query function	query_gpt helper established
Exp 2	Code Q&A — explain authenticate_user()	Correctly identified params, returns & side effects
Exp 3	Simplified ask_gpt wrapper	Reduced boilerplate for experiments
Exp 4	Onboarding — where to add email alerts?	Directed to /alerts/ folder + slack_alerts.py pattern
Exp 5	Debugging — memory crash scenario	Identified collect() as root cause + 3 fixes

Key Findings

Code Comprehension

Correctly extracted function purpose, params, return values without hallucination

Onboarding Guidance

Directed new dev to right folder and reference file in a single response

Debug Support

Identified root cause, explained failure, ranked 3 fix options by preference

Streamlit App: Live Demo

App Features

- Cached model loading

@st.cache_resource for one-time load

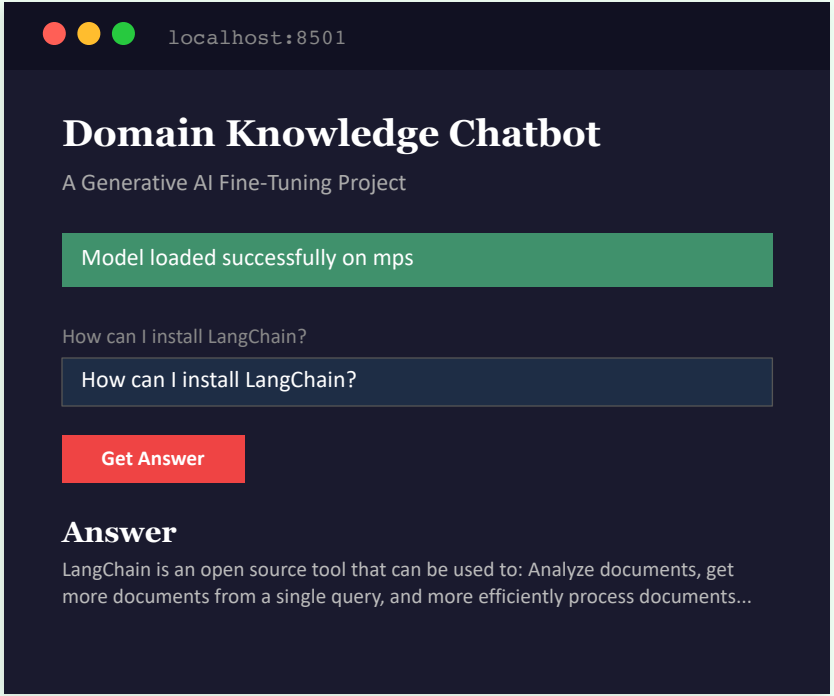
- Smart device detection

CUDA → MPS → CPU fallback chain

- Interactive sidebar controls

Token length & temperature sliders

- 5 pre-loaded sample questions
- Real-time text generation



Screenshot: Streamlit chatbot running on localhost with MPS device (Apple Silicon)

Ethics, Limitations & Conclusion

Ethical Considerations

- Privacy: Git data anonymized per GDPR
- IP: All data stays within org boundaries
- Hallucination: Responses cite source docs
- Overreliance: Positioned as complement, not replacement
- Equity: Coverage gaps tracked transparently

Limitations

- Sparse docs → weaker responses
- Model can still hallucinate on ambiguous context
- Context window limits fragment large files
- Data staleness if pipeline lags behind code
- Evaluation subjectivity across reviewers

Conclusion

The Domain Knowledge Chatbot is a technically validated solution to the institutional knowledge problem in software development. GPT-3.5-Turbo with RAG delivers accurate, context-grounded answers across code comprehension, onboarding, debugging, and design decisions. Response quality is directly tied to context quality — the system performs well with clean, well-structured documentation.

Reference: Bahree, A. (2023). Exploring GPT. Packt Publishing. | OpenAI. (2024). GPT-3.5-Turbo Fine-Tuning Guide.